NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

Nitte (DU) established under Section 3 of UGC Act 1956 | Accredited with 'A+' Grade by NAAC

Introduction To Python Programming

CS1001-2

Course Outcomes:

At the end of the course student will be able to

1. Understand and experiment with the basics of python programming like data types and looping.
2. Describe the concept of functions in python for solving real-world problems.
3. Apply the Python operations for manipulating strings, lists, tuples and dictionaries.
4. Demonstrate the proficiency in handling File Systems.
5. Understand the basic knowledge on Exception handling and plotting graphs.

Textbooks:

1. Kenneth A. Lambert, “The Fundamentals of Python: First Programs”, Cengage Learning, 2nd Edition, 2019.
2. Yashwanth Kanetkar, Adithya Kanetkar, “Let us Python”, 3rd Edition.
3. Mark Summerfield, “Programming in Python 3 - A Complete Introduction to the Python Language”, Second Edition, Addison-Wesley, 2009.
4. Y. Daniel Liang, “Introduction to Programming Using Python”, Pearson, 2013.

Unit -1
Introduction to Python and Function

Chapter 1:Python Concepts

Chapter 2: Design with functions

What is python?

Python is a very popular general-purpose interpreted, interactive, object-oriented, and high-level programming language.

Why to learn python?

- Python is Open Source which means its available free of cost.
- Python is simple and so easy to learn Python is versatile and can be used to create many different things.
- Python has powerful development libraries include AI, ML etc.

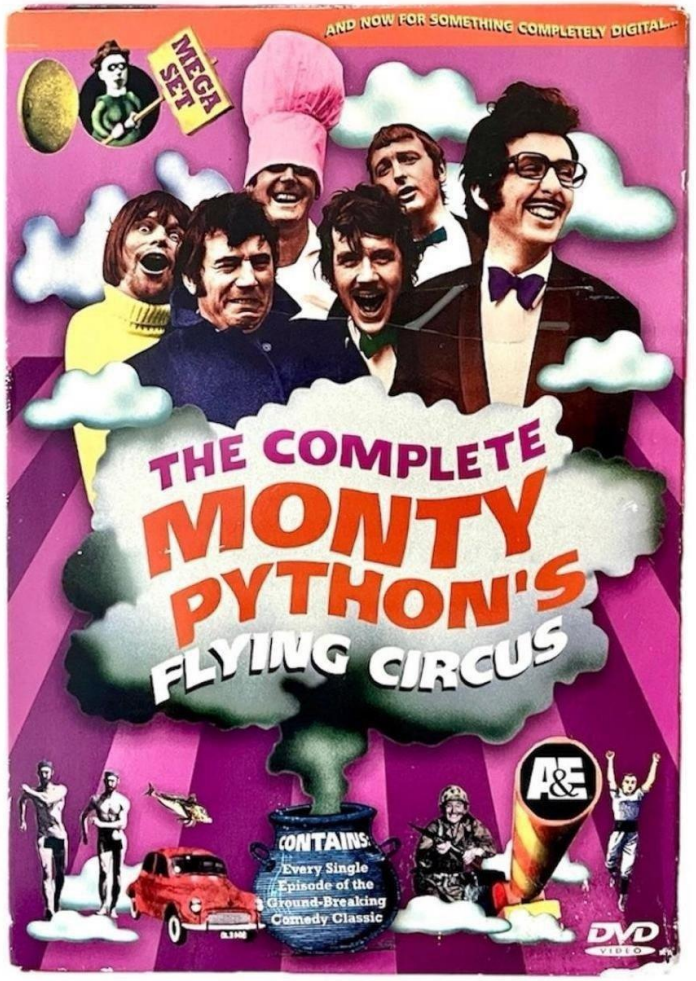
Python is a widely used high-level, interpreted programming language.

It was created by Guido van Rossum in 1991 and further developed by the Python Software Foundation

Why the name python?

Guido van Rossum was interested on watching a comedy show, which is telecasting on the BBC channel from 1969 to 1974 “**The complete Monty Python’s Flying Circus**”.

Guido Van Rossum thought he needed a name that was short, unique, and slightly mysterious for his programming language, so he decided to call the language **Python.**



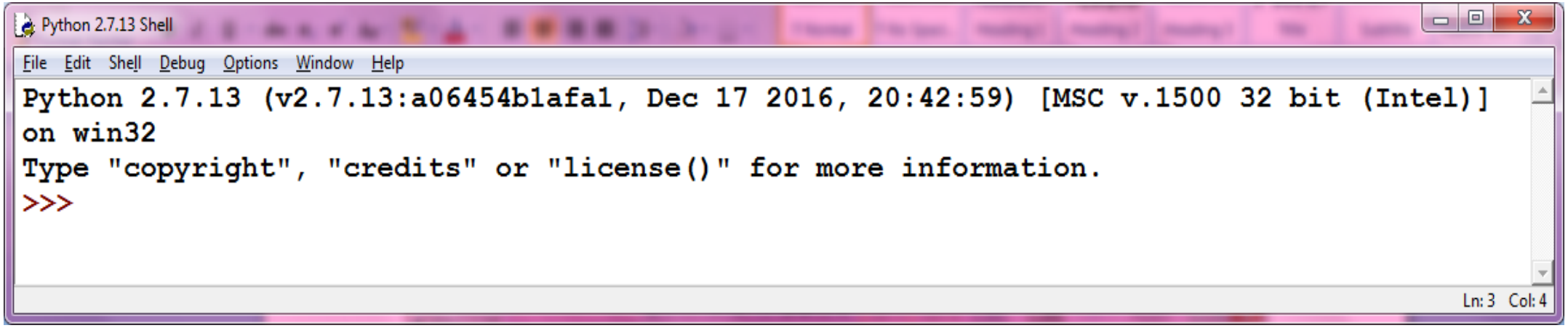
Key Features of Python

- **Python is Easy to Learn and Use:** There is no prerequisite to start Python, since it is Ideal programming language for beginners.
- **High Level Language:** Python don’t let you worry about low-level details, like memory management, hardware-level operations etc.
- **Python is Interpreted:** Code is executed line-by-line directly by interpreter, and no need for separate compilation. Which means –
  - You can run the same code across different platforms.
  - You can make the changes in code without restarting the program.

a) Demonstrate about Basics of Python Programming.

- Python comes with an interactive interpreter.
- When you type **python** in your shell or command prompt, the python interpreter becomes active with a

  >>> and waits for your commands.



Python 2.7.13 Shell

File Edit Shell Debug Options Window Help

```
Python 2.7.13 (v2.7.13:a06454b1afa1, Dec 17 2016, 20:42:59) [MSC v.1500 32 bit (Intel)]
on win32
Type "copyright", "credits" or "license()" for more information.
>>> print "hello"
hello
>>> print("hello")
hello
>>>
```

Ln: 7 Col: 4

ii. Running Python Scripts in IDLE (Integrated Development and Learning Environment):

- IDLE is the standard Python development environment.
- It's name is an acronym of **"Integrated DeveLopment Environment"**.
- It works well on both Unix and Windows platforms.
- It has a Python shell window, which gives you access to the Python interactive mode.

- Goto File menu click on New File (CTRL+N) and write the code and save add.py

iii. Running Python scripts in Command Prompt:

- Before going to run **python27** folder in the command prompt.
- Open your text editor, type the following text and save it as **hello.py.**
  `print "hello"`
- In the command prompt, and run this program by calling python hello.py.
- Make sure you change to the directory where you saved the file before doing it.

C:\Windows\system32\cmd.exe

```
C:\Python27>python Hello.py
Hello, Mothilal

C:\Python27>
```

Python Identifier

- Python Identifier is the name we give to identify a variable, function, class, module or other object.
- That means whenever we want to give an entity a name, that’s called identifier.

Python Keywords

- Python keywords are the words that are reserved.
- That means you can’t use them as name of any entities like variables, classes and functions.
- **There were 33 keywords in Python 3.7. However, the number increased to 35 in Python 3.8**.

False	def	if	raise
None	del	import	return
True	elif	in	try
and	else	is	while
as	except	lambda	with
assert	finally	nonlocal	yield
break	for	not	
class	from	or	
continue	global	pass	

Keyword	Description	Keyword	Description
and	A logical operator	from	To import specific parts of a module
as	To create an alias	global	To declare a global variable
assert	For debugging	if	To make a conditional statement
break	To break out of a loop	import	To import a module
class	To define a class	in	To check if a value is present in a list, tuple, etc.
continue	To continue to the next iteration of a loop	is	To test if two variables are equal
def	To define a function	lambda	To create an anonymous function
del	To delete an object	None	Represents a null value
elif	Used in conditional statements, same as else if	nonlocal	To declare a non-local variable
else	Used in conditional statements	not	A logical operator
except	Used with exceptions, what to do when an exception occurs	or	A logical operator
False	Boolean value, result of comparison operations	pass	A null statement, a statement that will do nothing
finally	Used with exceptions, a block of code that will be executed no matter if there is an exception or not	for	To create a for loop

Variable

- A variable, as the name indicates is something whose value is changeable over time.
- In fact a variable is a memory location where a value can be stored. Later we can retrieve the value to use.
- Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.
- Example

```
x = 5
y = "John"
print(x)
print(y)
```

- No need to define type of variable.
- During the execution, the type of a variable is inferred from the context in which it is being used. Hence Python is called **dynamically-typed language**.

- Simple variable assignment

```
a = 10
pi = 3.14
name = 'Sanjay'
```

```
x = 4              # x is of type int
x = “nitte"        # x is now of type str
print(x)
```

```
x = str(3)    # x will be '3‘
y = int(3)    # y will be 3
z = float(3)  # z will be 3.0
```

```
x = 5
y = "John"
print(type(x))
print(type(y))
```

Multiple variable assignment

```
a = 10 ; pi = 31.4 ; name = 'Sanjay'   # use ; as statement separator
a, pi, name = 10, 3.14, 'Sanjay'
a = b = c = d = 5
```

Rules for Writing Variable / Identifiers

There are some rules for writing Identifiers. But first you must know Python is case sensitive. That means **Name** and **name** are two different identifiers in Python. Here are some rules for writing Identifiers in python.

1. Identifiers can be combination of uppercase and lowercase letters, digits or an underscore (_). So **myVariable**, **variable_1**, **variable_for_print** all are valid python identifiers.
2. An Identifier can not start with digit. So, while **variable1** is valid, **1variable** is not valid.
3. We can’t use special symbols like !,#,@,%,$ etc in our Identifier.
4. Identifier can be of any length.

Whether the following statement is true?

x, y, z = “apple”, “banana”, “Cherry”

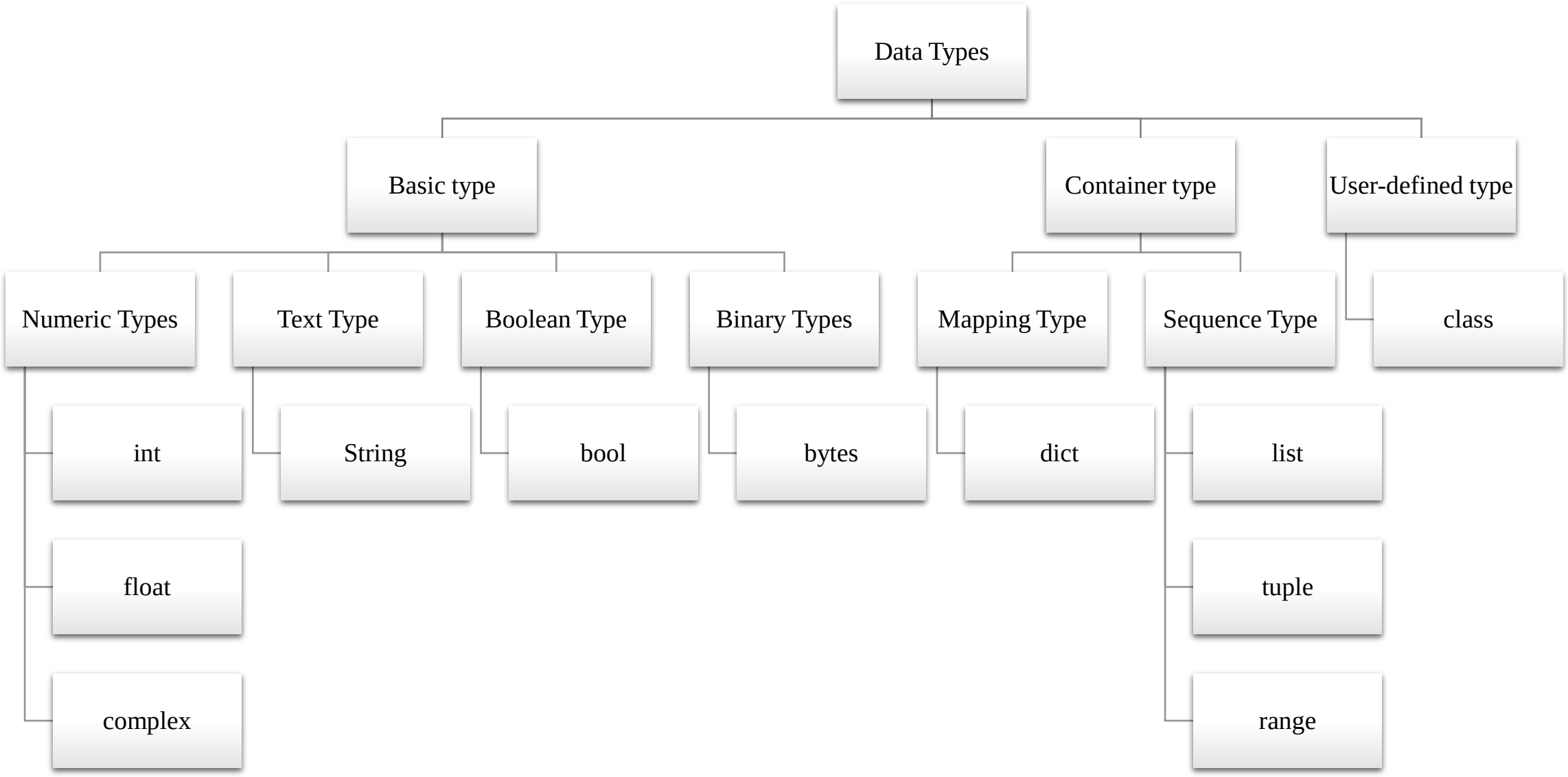
```
print(x)
print(y)
print(z)
```

Python Data Types

- Variables can store data of different types, and different types can do different things.
- Python has the following data types built-in by default, in these categories：
- 3 categories of data types:

  1. Basic types: int, float, complex, bool, string, bytes
  2. Container types: list, tuple, set, dict
  3. User-defined types: Class

Categories	
Text Type:	str
Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	dict
Boolean Type:	bool
Binary Types:	bytes, bytearray
None Type:	NoneType


Data Types
Basic type
Container type
User-defined type
Numeric Types
Text Type
Boolean Type
Binary Types
Mapping Type
Sequence Type
class
int
String
bool
bytes
dict
list
float
tuple
complex
range

- int can be expressed in binary, decimal, octal, hexadecimal.

Integer type	Starts with	Example
decimal	0-9	156
binary	0b/0B	0b101
octal	0o/0O	0o432
hexadecimal	0x/0X	0x443

- int can be of any arbitrary size
- Python has arbitrary precision integers. Hence, we can create as big integers as we want.
- Arithmetic operations can be performed on integers without worrying about overflow/underflow.

```
a = 123
b = 1234567890
c = 123456789012345678901234567890
```

- float can be expressed in fractional or exponential form
  - Eg: -314.1528, 3.141528e2, 3.141528E2
- complex contains real and imaginary part
  - 3+2j, 1+4J
- bool can take any of 2 Boolean values both starting in caps
  - True, False
- string is an immutable collection of Unicode characters enclosed within ‘ ‘, “ “ or “”” “”” “””

Input and Output

- **print()-** used to output values on the screen
- **input()** built-in function can be used to retrieve input values from keyboard.
- input() function returns a string
  - i.e if 23 is entered it returns ’23’
- So to perform arithmetic operations on the number entered . Convert string to int or float.

```
n = input('Enter your name: ')
age = int(input('Enter your age: '))
salary = float(input('Enter your salary: '))
print(name, age, salary)
```

Operators

Python divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators

1.Arithmetic operators

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponent	x ** y
//	Floor division	x // y

Operation Nuances

- On performing floor division, a//b, result is largest integer which is less than or equal to the quotient.

```
print(10 // 3)      # yields 3
print(-10 // 3)     # yields -4
print(10 // -3)     # yields -4
print(-10 // -3)    # yields 3
print(3 // 10)      # yields 0
print(3 // -10)     # yields -1
print(-3 // 10)     # yields -1
print(-3 // -10)    # yields 0
```

In -10 // 3, multiple of 3 which will yield -10 is -3.333, whose floor value is -4.

In 10 // -3, multiple of -3 which will yield 10 is -3.333, whose floor value is -4.

In -10 // -3, multiple of -3 which will yield -10 is 3.333, whose floor value is 3.

2. Comparison Operators

- These are used to compare two operands. For example, compare the ages of two persons, compare the prices of two or more items, etc.
- They result in either a TRUE (Non-zero) value or FALSE (Zero) value

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

3.Logical operators

- The result of these operators is either TRUE (ONE) or FALSE (ZERO)

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

4. Identity operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

```
x = ["apple", "banana"]
y = ["apple", "banana"]
z = x
```

- `print(x is z)` # returns True because z is the same object as x
- `print(x is y)` # returns False because x is not the same object as y, even if they have the same content
- `print(x == y)` # to demonstrate the difference between "is" and "==": this comparison returns True because x is equal to y

5.Membership operators

- Membership operators are used to test if a sequence is presented in an object

Operator	Description	Example	
in	Returns True if a sequence with the specified value is present in the object	x in y	x = ["apple", "grapes"] print("apple" in x) Output: TRUE
not in	Returns True if a sequence with the specified value is not present in the object	x not in y	x = ["apple", "grapes"] print("pineapple" not in x) Output: TRUE

6. Bitwise operators

- Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

- Take two bit values X and Y, where X = 7= $(111)_2$ and Y = 4 = $(100)_2$. Take Bitwise and of both X & y
- **Note:** Here, $(111)_2$ represent binary number.

$$\begin{array}{r} 111_2 \\ \& \; 100_2 \\ \hline 100_2 \; = \; 4 \end{array}$$

- Take two bit values X and Y, where X = 7= $(111)_2$ and Y = 4 = $(100)_2$. Take Bitwise OR of both X, Y

$$\begin{array}{r} 111_2 \\ | \; 100_2 \\ \hline 111_2 \; = \; 7 \end{array}$$

Take two bit values X and Y, where X = 7= (111)2 and Y = 4 = (100)2 . Take Bitwise and of both X & Y

$$
\begin{array}{r}
111_2 \\
\wedge\ 100_2 \\
\hline
011_2 \quad = 3
\end{array}
$$

Take two bit values X and Y, where X = 5= (101)2 . Take Bitwise NOT of X.

$$
\begin{array}{r}
\sim 1001_2 \\
\hline
0110_2 \quad = 6
\end{array}
$$

- **Example 1:**
  a = 5 = 0000 0101 (Binary)
  a << 1 = 0000 1010 = 10
  a << 2 = 0001 0100 = 20

  Example 2:
  b = -10 = 1111 0110 (Binary)
  b << 1 = 1110 1100 = -20
  b << 2 = 1101 1000 = -40

Example 1:
a = 10 = 0000 1010 (Binary)
a >> 1 = 0000 0101 = 5

Example 2:
a = -10 = 1111 0110 (Binary)
a >> 1 = 1111 1011 = -5

7.Assignment operators

- Assignment operators are used to assign values to variables

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3

Operator	Example	Same As
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Precedence and Associativity

- Precedence(priority)-multiple operators in an expression
- Associativity- operators with same precedence
- Left to right associativity or right to left associativity
- **c=a*b/c**
  - Same precedence
  - * is done before / (arithmetic operators have Left to right associativity)

Description	Operator	Associativity
Grouping	( )	Left to Right
Function call	function( )	Left to Right
Slicing	[start:end:step]	Left to Right
Exponentiation	**	Right to Left
Bitwise NOT	~	Right to Left
Unary plus / minus	+ -	Left to Right
Multiplication	*	Left to Right
Division	/	Left to Right
Modular Divsion	%	Left to Right
Addition	+	Left to Right
Subtraction	-	Left to Right
Bitwise left shift	<<	Left to Right
Bitwise right shift	>>	Left to Right
Bitwise AND	&	Left to Right
Bitwise XOR	^	Left to Right
Bitwise OR	|	Left to Right
Membership	In not in	Left to Right
Identity	is is not	Left to Right
Relational	< > <= >=	Left to Right
Equality	==	Left to Right
Inequality	!= <>	Left to Right
Logical NOT	not	Left to Right
Logical AND	and	Left to Right
Logical OR	or	Left to Right
Assignment	= += -= *= /= %= //= **= &= |= ^= >>= <<=	Right to Left

Type Conversions

- Mixed mode operations:
  - Operation between int and float will yield float.
  - Operation between int and complex will yield complex.
  - Operation between float and complex will yield complex.
- We can convert one numeric type to another using built-in functions.
  - **int() , float(), complex(), bool()**

- int() - constructs an integer number from an integer literal, a float literal (by removing all decimals), or a string literal (providing the string represents a whole number)

```
x = int(1)      # x will be 1
y = int(2.8)    # y will be 2
z = int("3")    # z will be 3
```

- float() - constructs a float number from an integer literal, a float literal or a string literal (providing the string represents a float or an integer)

```
x = float(1)    # x will be 1.0
y = float(2.8)  # y will be 2.8
z = float("3")  # z will be 3.0
w = float("4.2") # w will be 4.2
```

- str() - constructs a string from a wide variety of data types, including strings, integer literals and float literals

- x = str("s1") # x will be 's1'

  y = str(2) # y will be '2'

  z = str(3.0) # z will be '3.0’

Conversion	Explanation
int(float/numeric string)	from float/numeric string to int
int(numeric string, base)	from numeric string to int in base
float(int/numeric string)	from int/numeric string to float
float (int)	from int to float
complex(int/float)	Convert to complex with imaginary part zero
complex(int/float, int/float)	Convert to complex
bool(int/float)	from int to float to True/False
str(int/float/bool)	Convert to string
chr(int)	Yields a character corresponding to int

Built-in functions

- print() is also a built-in function.
- Help about any built-in function is available using **help(function)**
- **Built-in function commonly used with numbers are:**

```
abs(x)          # returns absolute value of x
pow(x, y)       # returns value of x raised to y
min(x1, x2,...) # returns smallest argument
max(x1, x2,...) # returns largest argument
divmod(x, y)    # returns a pair(x // y, x % y)
round(x [,n])   # returns x rounded to n digits after .
bin(x)          # returns binary equivalent of x
oct(x)          # returns octal equivalent of x
hex(x)          # returns hexadecimal equivalent of x
```

```
a = abs(-3)                    # assigns 3 to a
print(min(10, 20, 30, 40))     # prints 10
print(hex(26))                 # prints 1a
```

Built-in Modules

- Each module contains many functions.
  - math, cmath, random, decimal

math - many useful mathematics functions.

cmath - functions for performing operations on complex numbers.

random - functions related to random number generation.

decimal - functions for performing precise arithmetic operations.

- Mathematical functions in **math** module:

```
pi, e            # values of constants pi and e
sqrt(x)          # square root of x
factorial(x)     # factorial of x
fabs(x)          # absolute value of float x
log(x)           # natural log of x (log to the base e)
log10(x)         # base-10 logarithm of x
exp(x)           # e raised to x
trunc(x)         # truncate to integer
ceil(x)          # smallest integer >= x
floor(x)         # largest integer <= x
modf(x)          # fractional and integer parts of x
```

- **round()** built-in function can round to a specific number of decimal places, whereas
- **trunc(), ceil(),floor()** always round to zero decimal places.

- Trigonometric functions in **math** module:

```
degrees(x)      # radians to degrees
radians(x)      # degrees to radians
sin(x)          # sine of x radians
cos(x)          # cosine of x radians
tan(x)          # tan of x radians
sinh(x)         # hyperbolic sine of x
cosh(x)         # hyperbolic cosine of x
tanh(x)         # hyperbolic tan of x
acos(x)         # cos inverse of x, in radians
asin(x)         # sine inverse of x, in radians
atan(x)         # tan inverse of x, in radians
hypot(x, y)     # sqrt(x * x + y * y)
```

- Random number generation functions from **random** module:

```
random( )              # random number between 0 and 1
randint(start, stop)   # random number in the range
seed( )   # sets current time as seed for random number generation
seed(x)   # sets x as seed for random number generation logic
```

- To use functions present in a module, we need to import the module using **import** statement:

```
import math
import random
print(math.factorial(5))          # prints 120
print(math.degrees(math.pi))      # prints 180.0
print(random.random( ))           # prints 0.8960522546341796
```

- To get list of functions in each built-in module:

```
import math
print(dir(__builtins__))      # 2 underscores before and after builtins
```

```
print(dir(math))
```

Comments in python

What are Comments?

- Comments are used to explain the code and make it easier to read.
- Comments are ignored during execution.

- **Single line comment begin with #**

```
#Program to add 2 numbers
 a=b+c
```

- **Multiline comment written in a pair of ‘’ or “”””**

```
""" This function calculates the greatest common divisor (GCD) of two numbers using the Euclidean algorithm. The GCD of two numbers is the largest number that divides both of them without leaving a remainder. """
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
result = gcd(48, 18)
```

Indentation

- ‘Unexpected indent’

```
a = 20
    b = 45
```

Control statements

- So far statements in all our programs got executed sequentially or one after the other.
- Sequence of execution of instructions in a program can be altered using:
  - Decision control instruction
  - Repetition control instruction

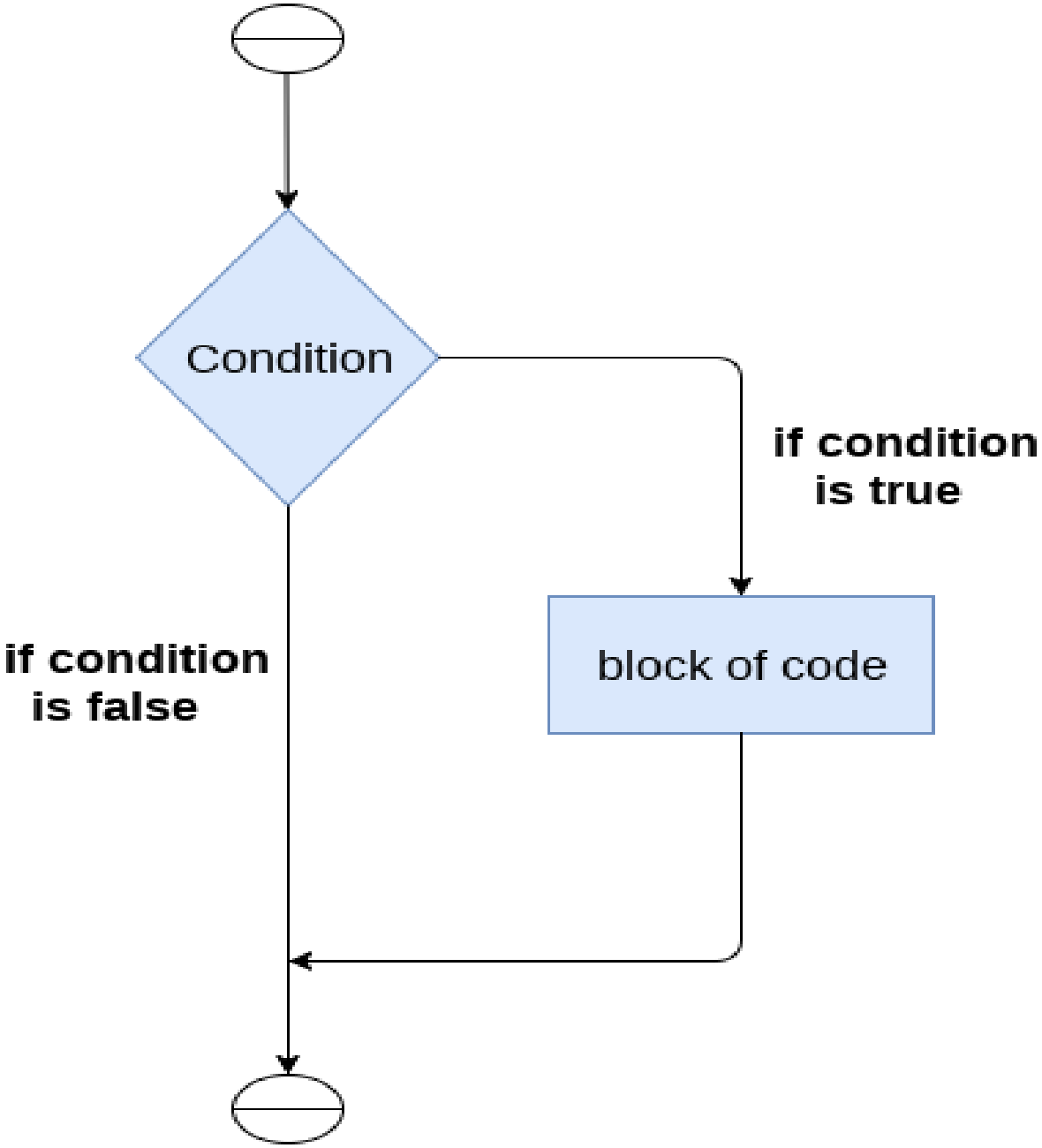
- Colon(:) after if, else, elif is compulsory.
- Statements in if block, else block, elif block have to be indented.
- Indented statements are treated as block of statements.
- Used to group statements. Use either 4 spaces or a tab for indentation.

If statement:

The if statement is used to test a specific condition. If the condition is true, a block of code (if-block) will be executed.

Syntax:

```
if condition:
    statement1
    statement2
statement3
```


Condition
if condition
is true
block of code
if condition
is false

Example:

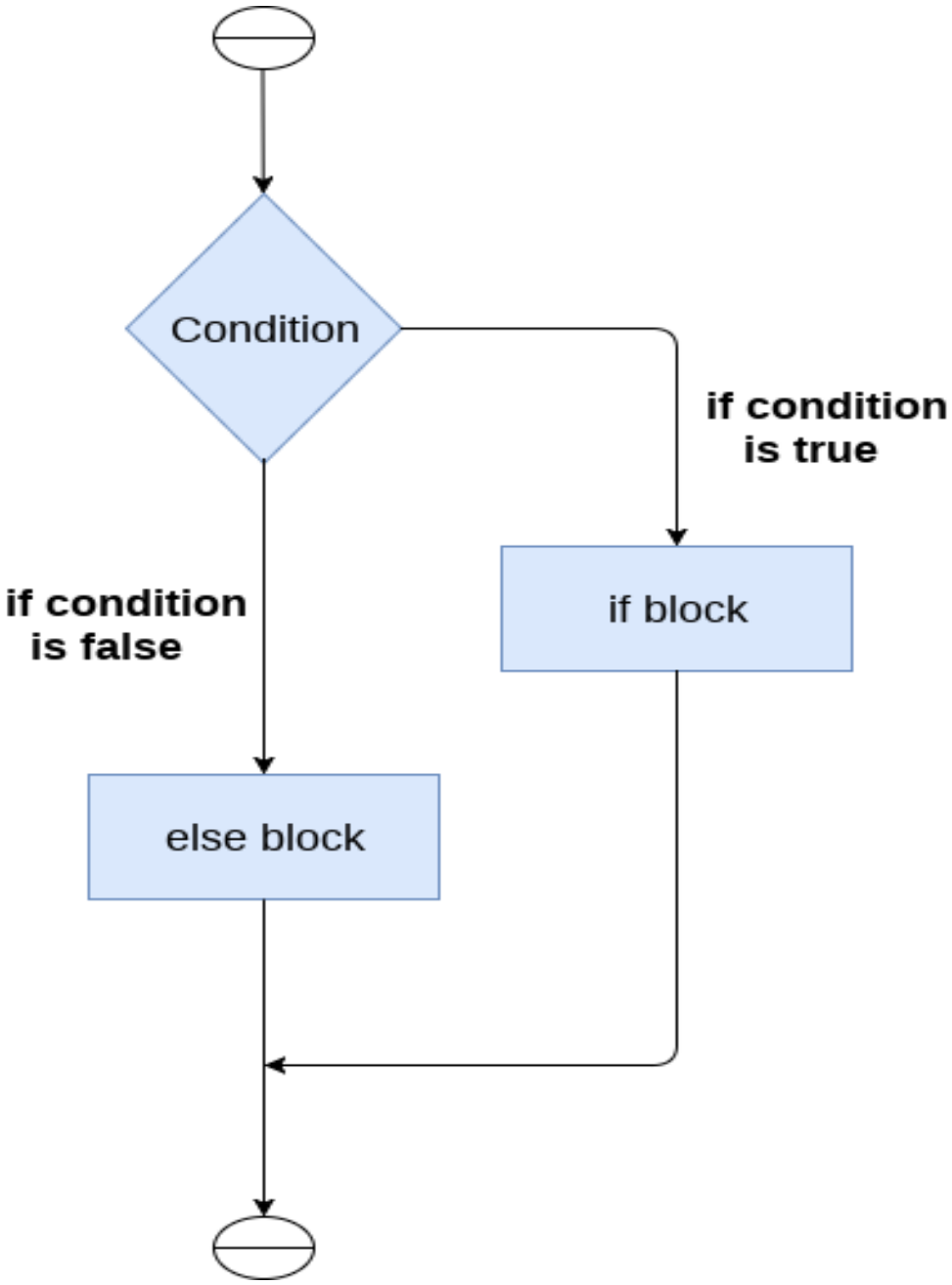
```
a = 30
b = 100
if b > a:
    print("b is greater than a")
print(“program ends here..”)
```

The if-else statement

The if-else statement is similar to if statement except the fact that, it also provides the block of the code for the false case of the condition to be checked. If the condition provided in the if statement is false, then the else statement will be executed.

Syntax:

```
if condition:
        statement1
        statement2
else:
        statement3
        statement4
```


Condition
if condition
is true
if block
if condition
is false
else block

Example:

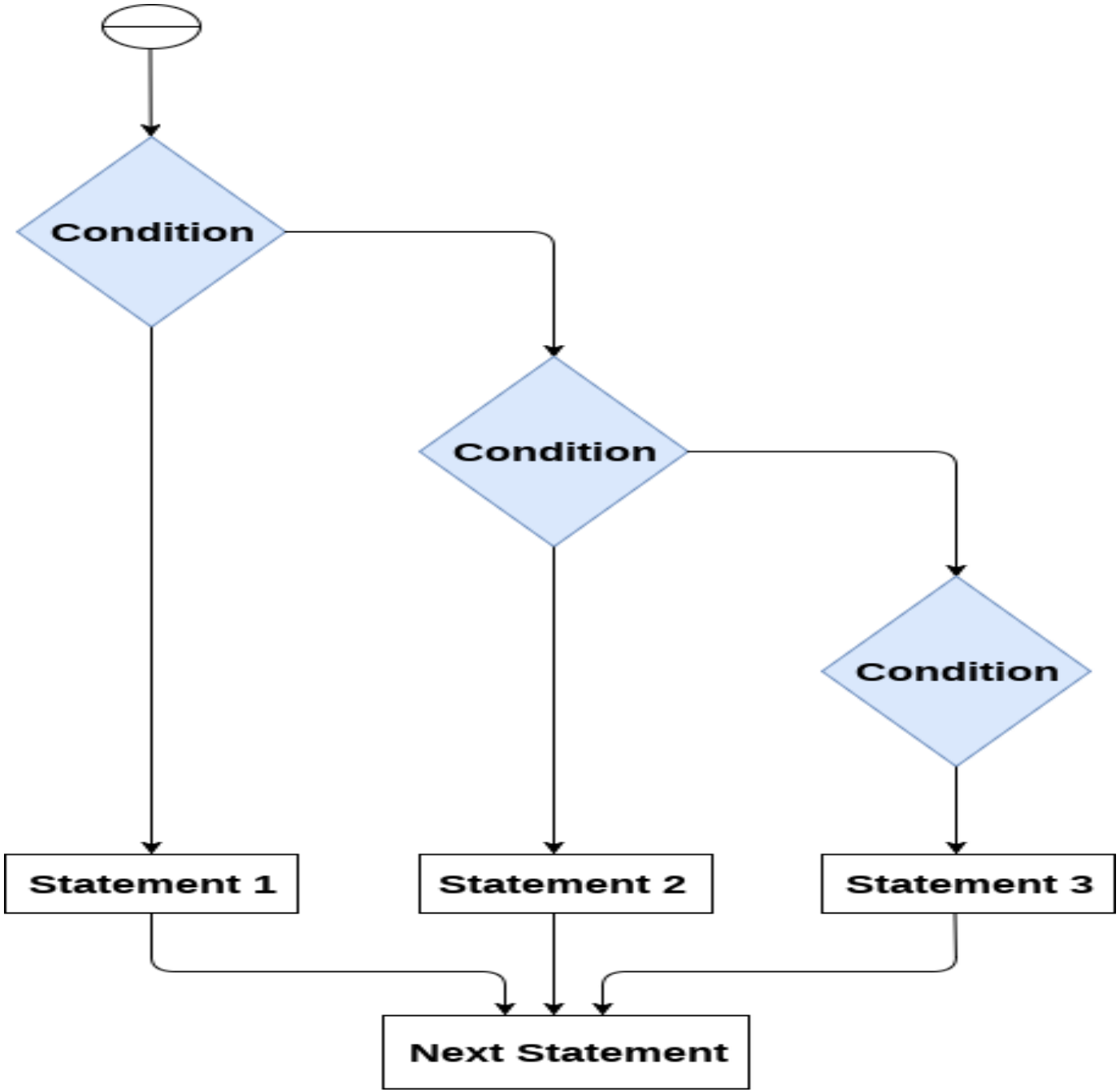
```
a = 30
b = 100
if b > a:
    print("b is greater than a")
else:
    print("b is greater than a")

print(“program ends here..”)
```

The elif statement:

- The elif statement enables us to check multiple conditions and execute the specific block of statements depending upon the true condition among them.
- We can have any number of elif statements in our program depending upon our need.

Syntax:

```
if condition1:
        statement1
        statement2
elif condition2 :
        statement3
elif condition3 :
        statement4
else :
        statement4
next statement
```


Condition
Condition
Condition
Statement 1
Statement 2
Statement 3
Next Statement

Example:

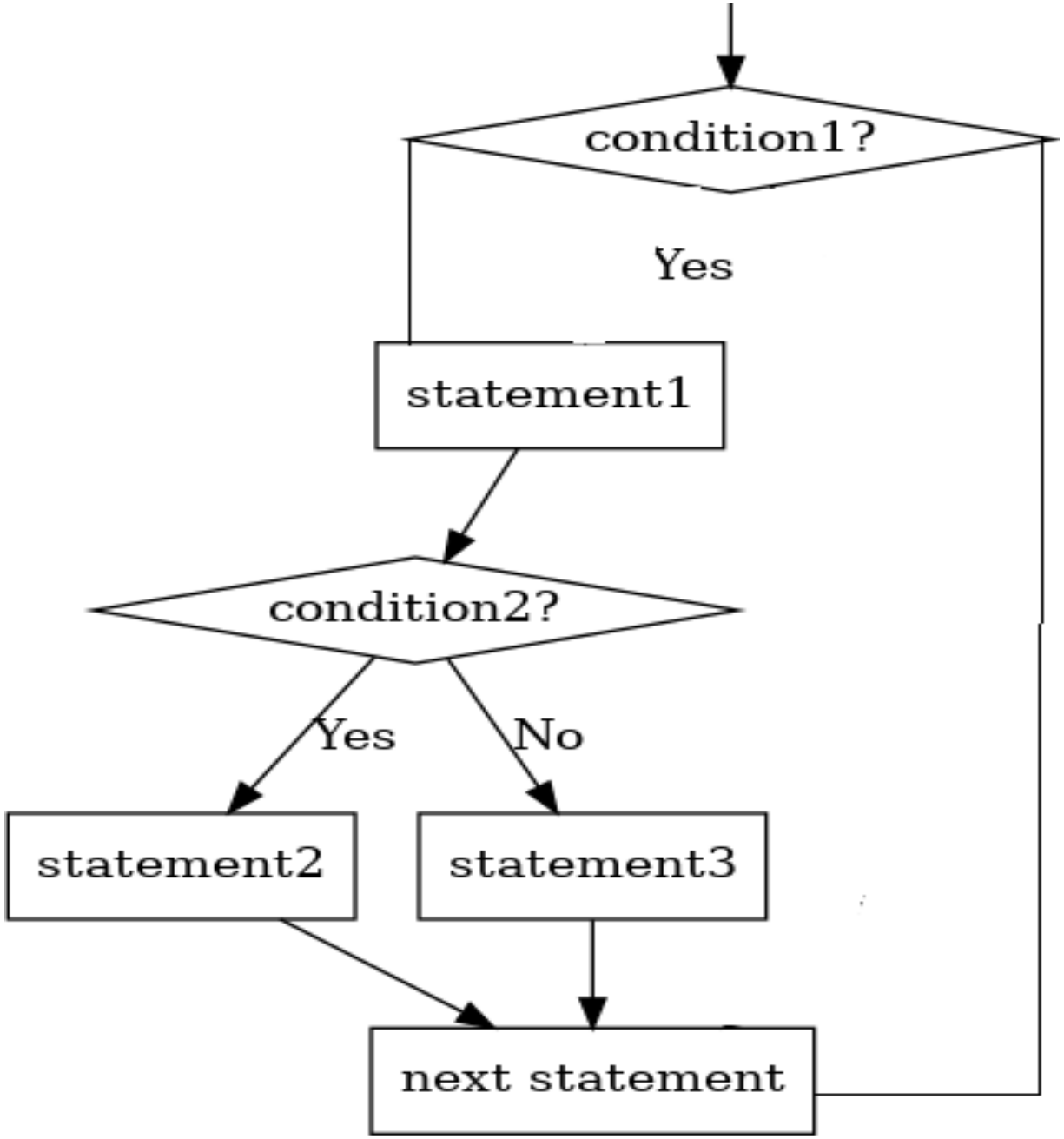
```
a = 30
b = 100
if a > b:
    print(“a is greater than b")
elif a < b:
    print("a is greater than b")
else:
    print("a and b are equal")
print(“thank you")
```

Nested if:

Python supports **nested if statements** which means we can use a conditional if and if...else statement inside an existing **if statement**.

Syntax:

```
if condition1:
        statement1
        if condition2:
                statement2
        else:
        statement3
next statement
```


condition1?
Yes
statement1
condition2?
Yes
No
statement2
statement3
next statement

Example:

```
num = 36
print ("num = ", num)
if num % 2 == 0:
        if num % 3 == 0:
                print ("Divisible by 3 and 2")
        else:
                print ("Divisible by 2 but not by 3")
print(“program ends here..”)
```

Repetition Control Instruction

- 2 types of repetition control instructions:
  - while
  - for

For loop:

for is used to iterate over elements of a sequence such as string, tuple or list.

Syntax:

```
for value in sequence:
        loop body
```

2 types:

```
for var in list :
    statement1
    statement2
```

```
for var in list :
    statement1
    statement2
else :
    statement3
    statement4
```

- During each iteration **var** is assigned the next value from the list.
- In place of a list, a string, a tuple, set or dictionary can also be used.
- **else** block is optional.
- **If present,**it is executed if loop is not terminated abruptly using break.

```
x = ‘nmamit’
for i in x:
   print(i)
```

output: ??

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
```

Output:

```
apple
banana
cherry
```

Usage of for loop

- A for loop can be used in the following two situations:
  - Repeat a set of statements a finite number of times.
  - Iterate through a string, list, tuple, set or dictionary.
- To repeat a set of statements a finite number of times a built-in function **range()** is used.

- **range()** function generates a sequence of integers:
  - range(10)- generates a number from 0 to 9
  - range(10,20)- generates a number from 10 to 19
  - range(10,20,2)- generates a number from 10 to 19 in step of 2
  - range(20,10,-3)- generates a number from 20 to 9 in step of -3
- range() cannot generate sequence of floats.

➢ **range(start, stop, step)**

- Produces a sequence of integers from start(inclusive) to stop(exclusive) by step.

- **Example 1:**

```
for x in range(2, 30, 3):
    print(x)
```

Output: 2 5 8 11 14 17 20 23 26 29

Example 2:

```
for i in range(5, -1, -1):
   print(i)
```

Output: 5 4 3 2 1 0

- The list of numbers generated using **range()** can be iterated through using a for loop.

```
for i in range(1, 10, 2) :
    print(i, i * i, i * i * i)
```

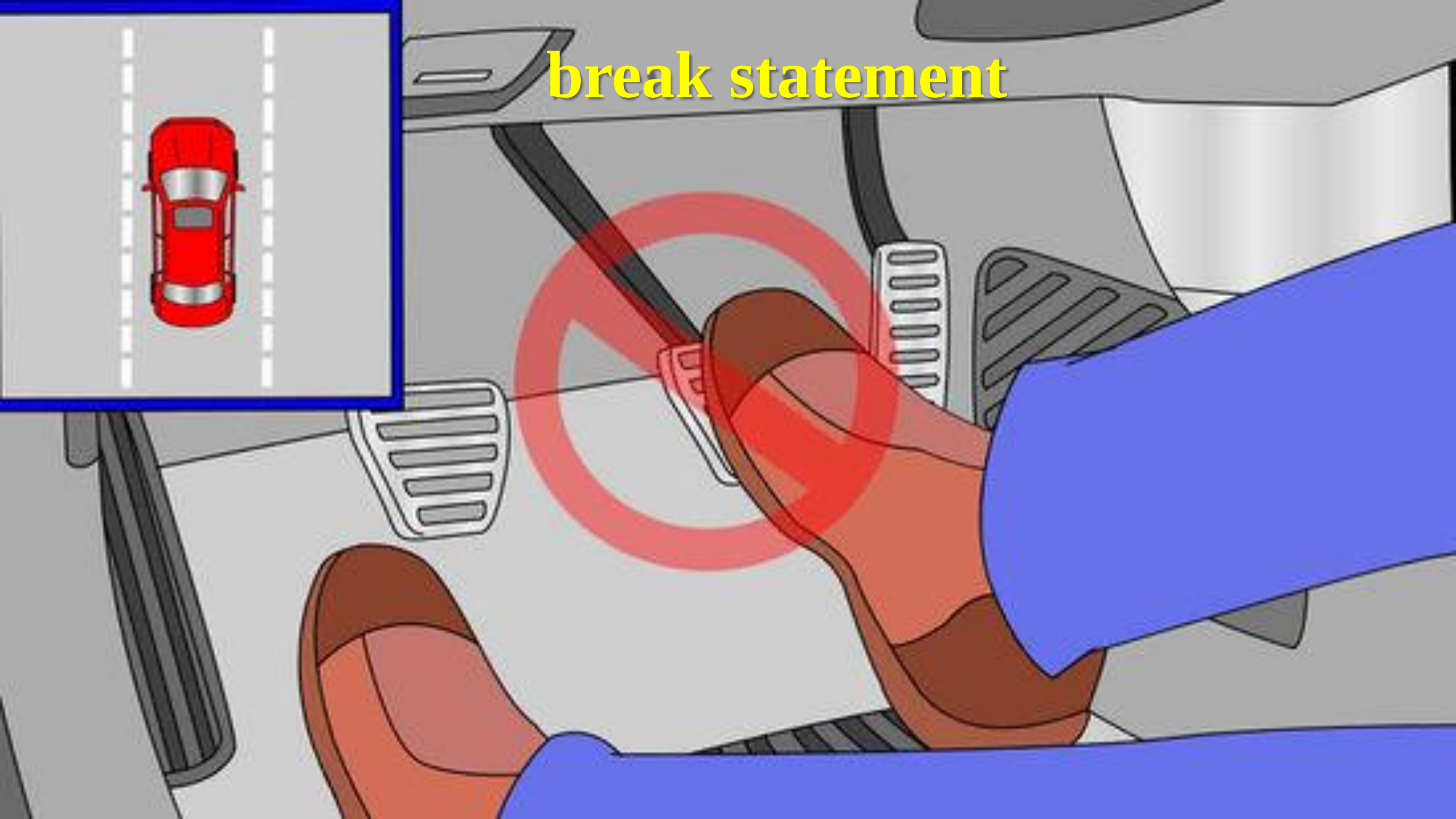
- Output:

```
1 1 1
3 9 27
5 25 125
7 49 343
9 81 729
```

- for can be used to iterate through a string, list, tuple, set or dictionary as shown below:

```
for char in 'Leopard' :
    print(char)
for animal in ['Cat', 'Dog', 'Tiger', 'Lion', 'Leopard'] :
    print(animal)
for flower in ('Rose', 'Lily', 'Jasmine') :
    print(flower)
for num in {10, 20, 30, -10, -25} :
    print(num)
for key in {'A101' : 'Rajesh', 'A111' : 'Sunil', 'A112' : 'Rakesh'} :
    print(key)
```

break statement



The break Statement

The break statement in Python is used to exit or “break” out of a loop (either a for or while loop) prematurely before the loop has iterated through all its items or reached its condition.

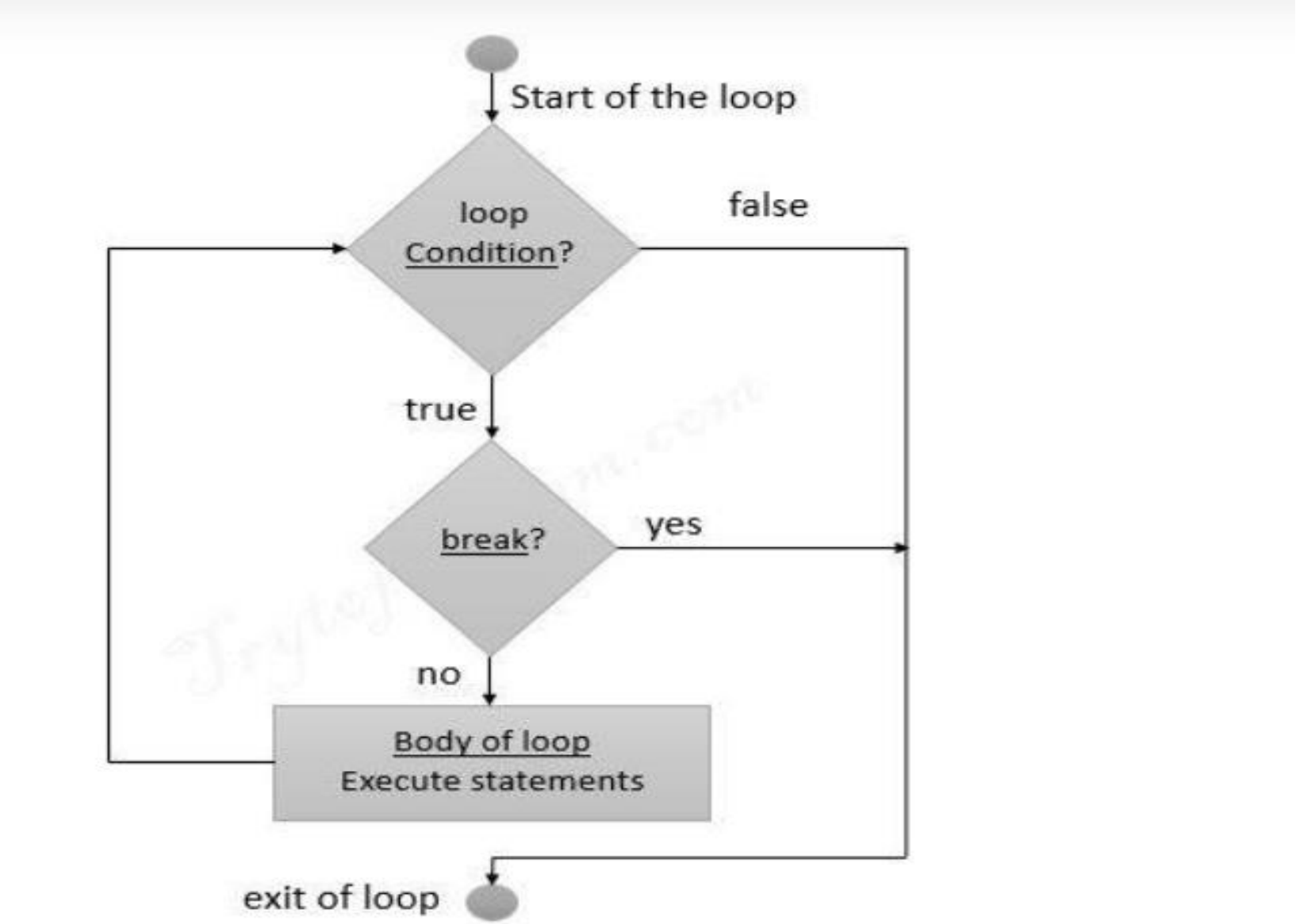
Example:

```
x = ‘nmamit’
for i in x:
    if i==‘i’:
        break
    print(i)
```

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

Output:

Working of break statement



The continue Statement

With the continue statement we can stop the current iteration of the loop, and continue with the next.

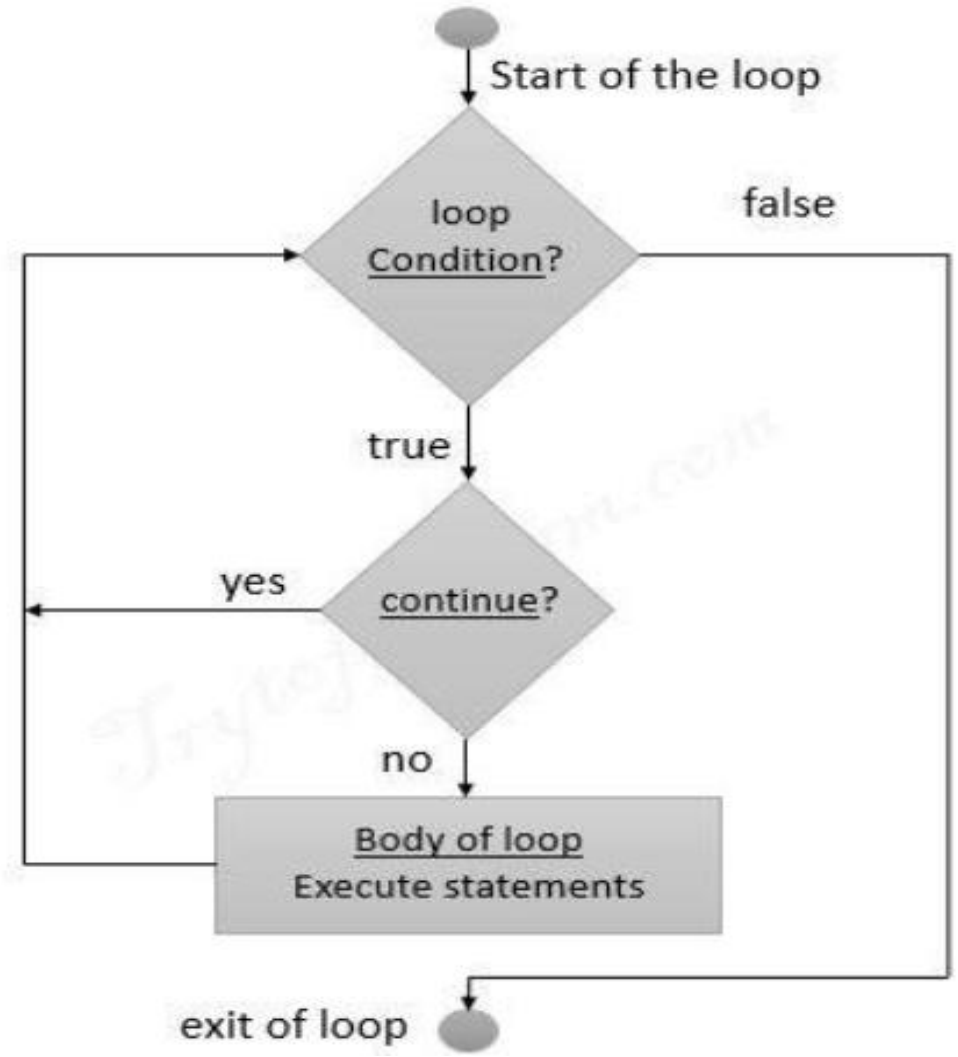
Python **continue statement** is used to skip the execution of the program block and returns the control to the beginning of the current loop to start the next iteration.

```
x = ‘nmamit’
for i in x:
    if i==‘i’:
        continue
    print(i)
```

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

Output:

Working of continue statement:



Using else Statement with for Loop

- A loop expression and an else expression can be connected in Python.
- After the circuit has finished iterating over the list, the else clause is combined with a for Loop.

Example:

```
for x in range(6):
  if x == 3:
        break
        print(x)
else:
        print("finished!")
```

```python
start = int(input("Enter starting range"))
end = int(input("Enter ending range"))
c=0
print("Prime numbers between", start, "and", end, "are:")

for num in range(start, end+1):
    # all prime numbers are greater than 1
    if num > 1:
        for i in range(2, num):
            if (num % i) == 0:
                break
        else:
            c=c+1
            print(num)
if(c==0):
            print("There are no prime numbers in the range")
```

while loop

A while loop in Python is a control flow statement used to repeat a block of code as long as a specified condition is True. It's especially useful when the number of iterations isn't known beforehand but depends on some dynamic condition.

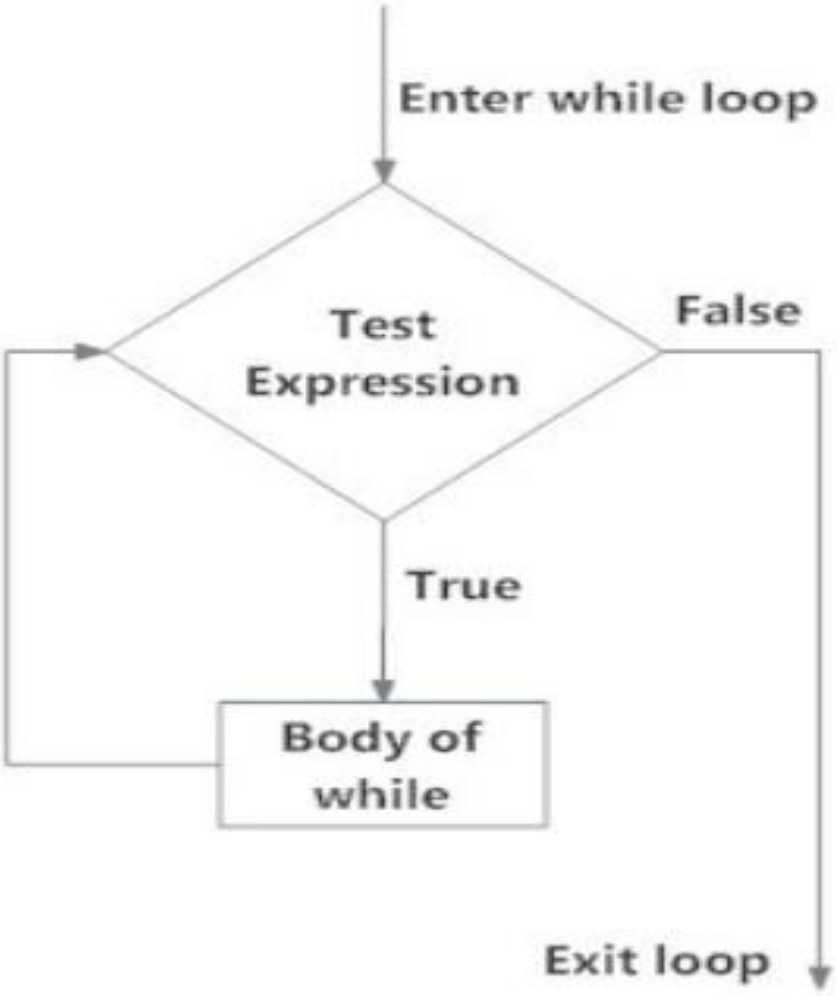
Syntax:

```
while expression:
        statement(s)
```

- **condition:** This is a boolean expression. If it evaluates to True, the code inside the loop will execute.

- **statement(s):** These are the statements that will be executed during each iteration of the loop.

Working of while loop



Snippet 1: Print Numbers from 1 to 5

```
num = 1
while num <= 5:
    print(num)
    num += 1
```

Output:

```
1
2
3
4
5
```

Snippet 2: Sum of Numbers from 1 to 10

Snippet 3: Even Numbers Between 1 and 10

Snippet 4: Print "Hello, World!" 5 Times using while

Snippet 6: Infinite Loop

```
while True:
    print("This will run forever!")
```

Output:

The string “This will run forever!” will be printed infinite times.

Syntax:

- Usage of break with for loop

```
for val in sequence:
  # code
  if condition:
    break

  # code
```

- Usage of break with while loop

```
while condition:
  # code
  if condition:
    break

  # code
```

Syntax:

- Usage of continue with for loop

```
for val in sequence:
    # code
    if condition:
        continue

    # code
```

- Usage of continue with while loop

```
while condition:
    # code
    if condition:
        continue

    # code
```

while loop with else

When the condition becomes false, the statement immediately after the loop is executed. The else clause is only executed when your while condition becomes false. If you break out of the loop, or if an exception is raised, it won't be executed

```
i = 0
while i < 4:
    i += 1
    print(i)
else:  # Executed because
no break in for
    print("No Break\n")
```

```
i = 0
while i < 4:
    i += 1
    print(i)
    break
else:  # Not executed as
there is a break
    print("No Break")
```

pass statement

The Python pass statement is a null statement. But the difference between pass and comment is that comment is ignored by the interpreter whereas pass is not ignored.

Syntax:

pass

What is pass statement in Python?

When the user does not know what code to write, So user simply places a pass at that line. Sometimes, the pass is used when the user doesn’t want any code to execute. So users can simply place a pass where empty code is not allowed, like in loops, function definitions, class definitions, or in if statements. So using a pass statement user avoids this error.

Using pass With Conditional Statement

```
n = 10
n=n+1
# use pass inside if statement
if n > 9:
    pass
else:
    print("Hello")
```

Output

Using pass With Conditional Statement

Output

```
Hello

=== Code Execution Successful ===
```

```
n = int(input('Enter a number: '))
if n > 0 :
    flag = True
    print(n * n)
elif n < 0 :
    flag = True
    print(n * n * n)
```

Answer

- This is misleading code. At a later date, anybody looking at this code may feel that flag = True should be written outside if and else.

- Better code will be as follows:

```
n = int(input('Enter a number: '))
if n > 0 :
    flag = True
    print(n * n)
elif n < 0 :
    flag = True
    print(n * n * n)
else :
    pass     # does nothing on execution
```

Using pass With Looping Statement

```python
n = 10
for i in range(n):

  # pass can be used as placeholder
  # when code is to added later
  pass
print(i)
```

Output

```
9

=== Code Execution Successful ===
```

Using pass With Looping Statement (while)

This image cannot currently be displayed.

This image cannot currently be displayed.

FUNCTIONS:

Function is a sub program which consists of set of instructions used to perform a specific task. A large program is divided into basic building▪blocks called function.

- **<u>Need For Function:</u>**

❖ When the program is too complex and large they are divided into parts. Each part is separately coded and combined into single program. Each subprogram is called as function.

❖ Debugging, Testing and maintenance becomes easy when the program is divided into subprograms.

❖ Functions are used to avoid rewriting same code again and again in a program.

❖ Function provides code re-usability

❖ The length of the program is reduced.

- **<u>Types of function:</u>**

Functions can be classified into two categories:

i) Built in function

ii) user defined function

i) Built in functions

❖ Built in functions are the functions that are already created and stored in python.

These built in functions are always available for usage and accessed by a programmer. It cannot be modified.

Built in function	Description
>>>max(3,4) 4	# returns largest element
>>>min(3,4) 3	# returns smallest element
>>>len("hello") 5	#returns length of an object
>>>range(2,8,1) [2, 3, 4, 5, 6, 7]	#returns range of given values
>>>round(7.8) 8.0	#returns rounded integer of the given number
>>>chr(5) \x05'	#returns a character (a string) from an integer
>>>float(5) 5.0	#returns float number from string or integer
>>>int(5.0) 5	# returns integer from string or float
>>>pow(3,5) 243	#returns power of given number
>>>type(5.6) <type 'float'>	#returns data type of object to which it belongs
>>>t=tuple([4,6.0,7]) (4, 6.0, 7)	# to create tuple of items from list
>>>print("good morning") Good morning	# displays the given object
>>>input("enter name: ") enter name : George	# reads and returns the given string

ii)**User Defined Functions:**

❖ User defined functions are the functions that programmers create for their requirement and use.

❖ These functions can then be **combined to form module** which can be used in other programs by importing them.

❖ Advantages of user defined functions:

- Programmers working on large project can divide the workload by making different functions.
- If repeated code occurs in a program, function can be used to include those codes and execute when needed by calling that function.

Function definition: (Sub program)

- def keyword is used to define a function.
- Give the function name after def keyword followed by parentheses in which arguments are given.
- End with colon (:)
- Inside the function add the program statements to be executed
- End with or without return statement

Syntax:

```
def fun_name(Parameter1,Parameter2…Parameter n):
  statement1
  statement2…
  statement n
  return[expression]
```

- **Example:**

```
def my_add(a,b):
  c=a+b
  return c
```

Function Calling:

- Once we have defined a function, we can call it from another function, program or even the Python prompt.
- To call a function we **simply type the function name with appropriate arguments.**

- Example:

```
x=5
y=4
my_add(x,y)
```

Flow of Execution:

- The order in which statements are executed is called the **flow of execution**
- Execution always begins at the first statement of the program
- Statements are executed one at a time, in order, from top to bottom.
- Function definitions do not alter the flow of execution of the program, but remember that statements inside the function are not executed until the function is called.
- Function calls are like a bypass in the flow of execution. Instead of going to the next statement, the flow jumps to the first line of the called function, executes all the statements there, and then comes back to pick up where it left off.

Note: When you read a program, don’t read from top to bottom. Instead, follow the flow of execution. This means that you will read the **def** statements as you are scanning from top to bottom, but you should skip the statements of the function definition until you reach a point where that function is called.

Parameters And Arguments:

Parameters(Formal Parameters):

· Parameters are the value(s) provided in the parenthesis when we write function header.

· These are the values required by function to work.

· If there is more than one value required, all of them will be listed in parameter list separated by **comma.**

· Example: def my_add**(a,b):**

Arguments(Actual Parameters) :

- Arguments are the value(s) provided in function call/invoke statement.
- List of arguments should be supplied in same way as parameters are listed.
- Bounding of parameters to arguments is done 1:1, and so there should be same number and type of arguments as mentioned in parameter list.
- Example: my_add(x,y)

Difference between Parameter and argument:

Aspect	Parameter	Argument
Definition	A variable listed in the function definition.	A value passed to the function when it is called.
Location	Appears in the function definition.	Appears in the function call.
Purpose	Acts as a placeholder for values that will be received.	Provides actual data for function execution.
Example	**def greet(name):** where name is a parameter.	**greet("Alice")** "Alice" is an argument.
Types	Positional, Default, Variable-Length	Positional, Keyword, Default, and Variable-Length arguments.

RETURN STATEMENT:

- The **return statement is used to exit a function** and go back to the place from where it was called.
- If the return statement has no arguments, then it will not return any values. But exits from function.

- **Syntax:**

  return[expression]

- **Example:**

```
def my_add(a,b):
  c=a+b
  return c
x=5
y=4
print(my_add(x,y))
```

- **Output:**

9

The pass Statement

The pass statement serves as a placeholder for future code, preventing errors from empty code blocks.

```
def function():
        pass                # this will execute without any action
or error
function()
```

ARGUMENTS TYPES:

1. Required Arguments
2. Keyword(named) Arguments
3. Default Arguments
4. Variable length Arguments

1. Required Arguments:

The number of arguments in the function call should match exactly with the function definition.

```
def my_details( name, age ):

print("Name: ", name)

print("Age ", age)

return

my_details("george",56)
```

- **Output:**

Name: george

Age 56

2. Keyword(named) Arguments:

- Python interpreter is able to use the keywords provided to match the values with parameters even though if they are arranged in out of order.

```
def my_details( name, age ):
print("Name: ", name)
print("Age ", age)
return
my_details(age=56,name="george")
```

- **Output:**

Name: george
Age 56

3. Default Arguments:

- Assumes a default value if a value is not provided in the function call for that argument.

```
def my_details( name, age=40 ):
print("Name: ", name)
print("Age ", age)
return
my_details(name="george")
```

- **Output:**

Name: george
Age 40

4. Variable length Arguments

- If we want to specify more arguments than specified while defining the function, variable length arguments are used.
- It is denoted by * symbol before parameter.

```
def my_details(*name ):
print(*name)
my_details("rajan","rahul","micheal",ärjun")
```

- **Output:**

rajan rahul micheal ärjun

Positional-Only Arguments

- You can specify that a function can have ONLY positional arguments, or ONLY keyword arguments.
- To specify that a function can have only positional arguments, add , / after the arguments:

Example

```
def my_function(x, /):
  print(x)


my_function(3)
```

```
def my_function(x, /):
  print(x)

my_function(3)
```

```
def
my_function(x):
  print(x)

my_function(x = 3)
```

```
//error
def my_function(x, /):
  print(x)

my_function(x = 3)
```

Recursive Function

A Python function can be called from within its body. When we do so it is called a recursive function.

```
def fun( ) :
    # some statements
    fun( )    # recursive call
```

Recursive call keeps calling the function again and again, leading to an infinite loop.

A provision must be made to get outside this infinite recursive loop. This is done by making the recursive call either in if block or in else block as shown below:

```
def fun( ) :
    if condition :
        # some statements
    else
        fun( )    # recursive call
```

```
def fun( ) :
    if condition :
        fun( )
    else
        # some statements
```

- Variables created in a function die when control returns from a function.
- Recursive function may or may not have a return statement.
- Typically, during execution of a recursive function many recursive calls happen, so several sets of variables get created. This increases the space requirement of the function.
- Recursive functions are inherently slow since passing value(s) and control to a function and returning value(s) and control will slow down the execution of the function.
- Recursive calls terminate when the base case condition is satisfied.

Recursive Factorial Function

```
def refact(n) :
    if n == 0 :
        return 1
    else :
        p = n * refact(n - 1)
    return p

num = int(input('Enter any number: '))
fact = refact(num)
print('Factorial value = ', fact)
```

Lambda Function

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.

Python Lambda Functions are anonymous functions means that the function is without a name. As we already know the *def* keyword is used to define a normal function in Python. Similarly, the *lambda* keyword is used to define an anonymous function in Python.

- lambda *arguments* : *expression*

- x = lambda a : a + 10
  print(x(5))

- ```
  def myfun(n):
    return lambda a : a * n
  ```

- ```
  def myfun(n):
    return lambda a : a * n

  mydoubler = myfun(2)

  print(mydoubler(11))
  ```

- **MODULES:**

- **A module is a file containing Python definitions, functions, statements and instructions.**
- Standard library of Python is extended as modules.
- **To use these modules in a program, programmer needs to import the module.**
- Once we import a module, we can reference or use to any of its functions or variables in our code.

  - There is large number of standard modules also available in python.
  - Standard modules can be imported the same way as we import our user-defined modules.
  - Every module contains many function.
  - *To access one of the function , you have to specify the name of the module and the name of the function separated by dot .* **This format is called dot notation.**

- **Syntax:**

```
import module_name
module_name.function_name(variable)
```

- **Importing Builtin Module:**

```
import math
x=math.sqrt(25)
print(x)
```

- **Importing User Defined Module:**

```
import cal
x=cal.add(5,4)
print(x)
```

There are **four ways to import a module** in our program, they are

Import: It is simplest and most common way to use modules in our code.
Example:
`import math`
`x=math.pi`
`print("The value of pi is", x)`
Output: The value of pi is 3.141592653589793
	from import : It is used to get a specific function in the code instead of complete file.
	Example:
	`from math import pi`
	`x=pi`
	`print("The value of pi is", x)`
	Output: The value of pi is 3.141592653589793

import with renaming:
We can import a module by renaming the module as our wish.
Example:
`import math as m`
`x=m.pi`
`print("The value of pi is", x)`
Output: The value of pi is 3.141592653589793
	import all:
	We can import all names(definitions) form a module using *
	Example:
	`from math import *`
	`x=pi`
	`print("The value of pi is", x)`
	Output: The value of pi is 3.141592653589793

- Built-in python modules are,

1.math – mathematical functions:

- some of the functions in math module is,
- math.ceil(x) - Return the ceiling of *x*, the smallest integer greater than or equal to *x*
- math.floor(x) - Return the floor of x, the largest integer less than or equal to x.
- math.factorial(x) -Return x factorial.
- math.gcd(x,y)- Return the greatest common divisor of the integers a and b
- math.sqrt(x)- Return the square root of x
- math.log(x)- return the natural logarithm of x math.log10(x) – returns the base-10 logarithms math.log2(x) - Return the base-2 logarithm of x. math.sin(x) – returns sin of x radians
- math.cos(x)- returns cosine of x radians
- math.tan(x)-returns tangent of x radians
- math.pi - The mathematical constant π = 3.141592 math.e – returns The mathematical constant e = 2.718281

Thank You